

School of Computing and Engineering
Higher National Diploma in Network
engineering
(HNDNE25.2)



Red-Black Tree Based Dynamic Flow
Management in IDS/IPS Systems

Data Structure and Algorithms
Module Course work
National Institute of Business Management

Group members:

COHNDNE252F-004 K.A Baylon

COHNDNE252F-005 S.M.J Nawarathna

COHNDNE252F-007 J A K C Jayasinghe

Contents

Abstract	3
Introduction	4
Problem Statement	5
Overview of Red-Black Tree	6
Justification for Selecting Red-Black Tree	7
Conceptual Design and Operation	8
Complexity Analysis	10
Advantages and Limitations	12
Advantages.....	12
Limitations	13
Conclusion	14
GitHub Repository link.....	14

Abstract

Modern networks generate enormous volumes of constantly changing traffic. To identify malicious behavior in real time, Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) must efficiently track, update, and remove thousands of active flows with minimal delay.

This report explores the use of a Red-Black Tree as a data structure for managing dynamic network sessions in IDS/IPS environments. It explains why this structure is well suited to the task and outlines how it supports real-time flow tracking and anomaly detection.

Introduction

Network security is now a major concern across enterprise and cloud infrastructures. IDS and IPS solutions continuously analyze traffic to detect threats such as unauthorized access, denial-of-service attacks, port scanning, and lateral movement.

To function effectively, these systems maintain records of active network flows. Since flows are created and terminated continuously, the environment is highly dynamic. Efficient flow management is therefore essential for preserving performance and maintaining detection accuracy.

This report examines how a Red-Black Tree can be applied to address these challenges and help maintain credibility with IDS/IPS systems.

Problem Statement

A network flow is commonly identified using a five-tuple / header consisting of the source IP address, destination IP address, source port, destination port, and protocol.

Each packet entering an IDS must be associated with its corresponding flow key so that the system can maintain an accurate view of ongoing communication sessions. Once a match is found, the IDS will update packet statistics, evaluate traffic behavior, detect potential anomalies, and remove flows that have expired or become inactive.

This process can be challenging with modern networks generating high volumes of traffic, where flows are continuously created and terminated. Our system must therefore handle frequent insertions and deletions while still delivering consistent and predictable performance. Any inefficiency may lead to processing delays, packet loss, or even system overload which can compromise the performance in the internal network. Therefore, we can consider using a suitable data structure to manage this type of inputs which also allows easy manipulation to suit the basic requirements.

Overview of Red-Black Tree

A Red-Black Tree is a self-balancing binary search tree that maintains balance using node coloring and tree rotations. Each node is either red or black, and specific rules ensure the height of the tree remains approximately balanced.

Key characteristics include:

- $O(\log n)$ search complexity
- $O(\log n)$ insertion complexity
- $O(\log n)$ deletion complexity
- Predictable worst-case performance

Compared to a standard Binary Search Tree, a Red-Black Tree prevents degeneration into a linear structure, ensuring consistent efficiency even under dynamic conditions.

Features of a RBT

- Self balancing BST
- Each node is either red or black
- Adjacent nodes cannot be red
- Root of the tree is always black
- Leaf nodes are always black
- Every path to a leaf node from the root should have equal no of black nodes
- Uses rotation and recoloring to balance the tree
- Height constrains $2 * \log_2(n + 1)$

Justification for Selecting Red-Black Tree

The selection of Red-Black Tree is justified based on IDS/IPS operational requirements:

1. Frequent Updates

Network flows are continuously created and removed. Red-Black Trees efficiently handle frequent insertions and deletions while maintaining balance.

2. Predictable Performance

Security systems cannot tolerate worst-case delays. Unlike hash tables, which may degrade under collision attacks, Red-Black Trees provide guaranteed $O(\log n)$ performance. Ensuring performance fluctuations at a constant rate.

3. Efficient Searching

Each incoming packet requires a flow lookup. The logarithmic search complexity ensures scalable performance.

4. Balanced Maintenance Cost

Compared to AVL trees, Red-Black Trees require fewer rotations during updates, making them suitable for highly dynamic environments such as IDS/IPS systems.

Conceptual Design and Operation

1. Flow Insertion

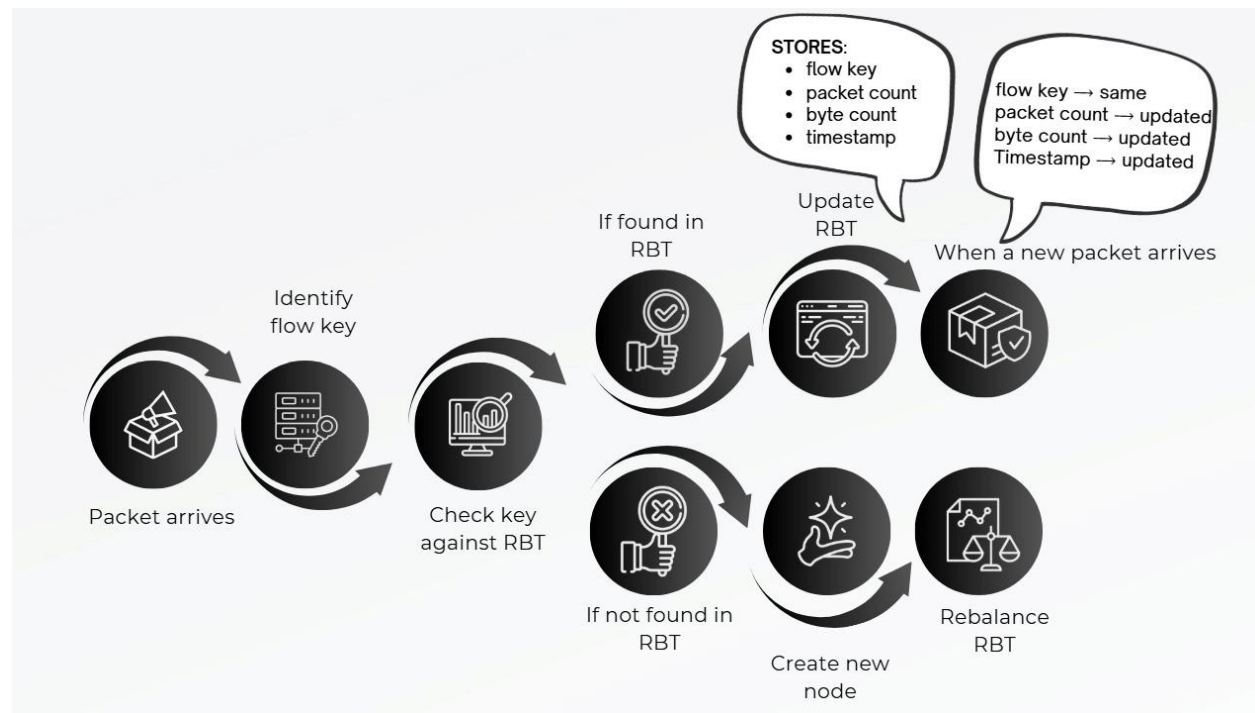
When a packet arrives:

- The system extracts the flow key.
- The Red-Black Tree is searched.
- If the flow does not exist, a new node is inserted.
- Rebalancing operations maintain tree properties.

2. Flow Update

If the flow exists:

- Packet count is incremented.
- Byte count is updated.
- Timestamp is refreshed.
- Behavioral thresholds are evaluated.



3. Intrusion Detection Logic

Each flow node stores:

- Packet count
- Byte count
- Session duration
- Last activity time

The IDS analyzes these parameters to detect anomalies such as:

- Sudden traffic bursts (possible DoS)
- Excessive connection attempts (port scanning)
- Abnormal session durations

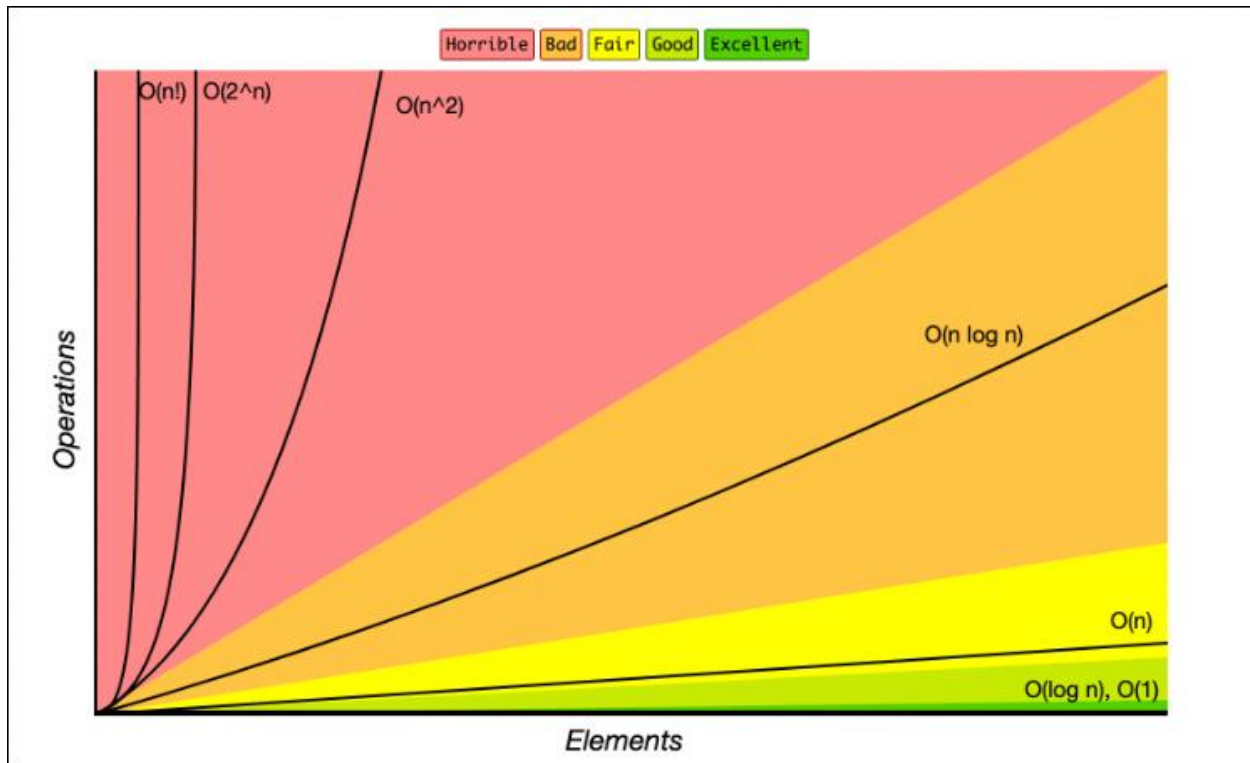
If malicious behavior is detected, the IPS component blocks the flow and may remove it from the tree.

Statistic	Normal behavior	Attack behavior	Type of attack
Packet count	50 - 200	50,000+ in secs	DoS attack
Byte count	Few KB or MB	Excessive size data transfer	Exfiltration attack
Session duration	Few seconds	Several hours	Malicious behavior
Number of flow keys	Few keys from single IP source	Same source IP 100+ new flow keys	Port scanning

4. Flow Expiration

Idle flows are periodically removed using delete operations. The Red-Black Tree maintains balance after deleting, preventing performance degradation. Minimizes wastage of resources.

Complexity Analysis



Time Complexity Analysis

The performance of an IDS/IPS system depends on how quickly it can process each incoming packet. Since every packet can be matched to an existing network flow or inserted as new flow, the time complexity of these operations should be sound.

In a Red-Black Tree, the height of the tree is always bound by:

$$h \leq 2 \log_2(n + 1)$$

where n is the number of nodes (active flows).

This guarantees that the tree remains approximately balanced even under continuous insert and delete operations.

Search Complexity – $O(\log n)$

Whenever a packet arrives, the system must locate its corresponding flow. Because the RBT remains balanced, the search time stays logarithmic even in the worst case. Therefore, this predictable lookup time prevents performance slowdowns as the number of active flows grows even during high traffic inflows.

Insertion Complexity – $O(\log n)$

When the system encounters a new flow, a node is added using standard binary search tree rules. After insertion, the tree performs adjustments by rotations and recoloring to maintain balance.

Compared to AVL trees, Red-Black Trees typically require fewer rotations, making them more efficient in environments where new flows are created frequently.

Deletion Complexity – $O(\log n)$

Flows must be removed when they expire or identified as malicious. Deleting a node may trigger recoloring and a few rotations to restore the tree's properties. These adjustments are limited and controlled. In IDS systems where flows are constantly expiring, this stable $O(\log n)$ deletion performance helps maintain consistent system behavior

Performance Under Traffic Spikes

One of the most critical aspects of IDS/IPS systems is behavior during attack scenarios such as:

- Distributed Denial of Service (DDoS)
- Port scanning floods
- SYN flood attacks

During such events:

- Thousands of new flows may be generated rapidly.
- Frequent insertions occur.
- Expired flows must be removed continuously.

This keeps the structure resilient against performance-based evasion attacks.

Advantages and Limitations

Advantages

Predictable Worst-Case Performance

Red-Black Trees provide guaranteed predictable performance even at worst case which is an advantage for security systems where worst-case degradation could allow attackers to bypass detection mechanisms.

Efficient Handling of Dynamic Updates

Red-Black Trees are specifically optimized for environments with frequent insertions and deletions, making them well-suited for real-time traffic management.

Scalability

As the number of active flows increases, the logarithmic height ensures that performance degrades slowly and predictably. This supports deployment in enterprise and cloud-scale environments.

Balanced Rotation Overhead

Red-Black Trees sacrifice strict balance for fewer rotations, making them more efficient under high update rates.

Ordered Traversal Capability

Because it's a Binary Search Tree, flows remain sorted.

This allows efficient range queries, statistical analysis, threshold-based anomaly detection and time-based flow expiration scans

This ordered structure is not naturally available in hash tables.

Resistance to Collision Attacks

Red-Black Trees use comparisons to organize and locate data, which makes them naturally resistant for this type attack and helps maintain stable performance.

Limitations

Implementation Complexity

Red-Black Trees are more difficult to be implement than simpler data structures. They require careful handling of node colors, rotations, and strict balancing rules. If not applied correctly, the tree can lose its structure, which may lead to errors or unstable behavior. Eventually, development and maintenance could be more challenging.

Slightly Higher Memory Usage

Each node in a Red-Black Tree stores extra information, such as pointers to nodes, and a color value. This means it uses more memory compared to simpler structures like arrays or linked lists. However, the additional memory cost usually acceptable given the performance benefits.

Slower Average Case Compared to Hash Tables

Red-Black Trees guarantee balanced performance rates. The trade-off is between faster average speed and guaranteed worst-case stability. In security systems, predictable performance is often more important than peak speed.

Cache Locality Considerations

Because tree nodes are scattered in memory, they may not utilize the CPU cache as efficiently as arrays, where data is stored contiguously. This can slightly reduce performance in extremely high-speed processing scenarios.

Conclusion

Managing dynamic network flows is a fundamental requirement in IDS/IPS systems. Due to the huge volume and unpredictability of network traffic, an efficient and stable data structure is necessary.

This report demonstrated that Red-Black Trees provide predictable performance for search, insertion, and deletion operations. Their ability to maintain balance under frequent updates makes them particularly suitable for dynamic intrusion detection environments.

Therefore, Red-Black Trees represent an effective and scalable solution for managing active network sessions in modern network security systems in comparison to other data structures.

GitHub Repository link

<https://github.com/Kendra-Bay/DSA-coursework.git>